

1 ▾ `/*Pointers in C++`

2 Pointers are variables that are used to
 store the address. Pointers are created
 using `"*"`. An example program of pointers
 is shown below*/

3

4 `#include<iostream>`

5 `using namespace std;`

6

7 ▾ `int main(){`

8 `// Basic Example`

9 `int a = 4;`

10 `int* ptr = &a;`

11 `cout<<"The value of a is "<<*(ptr)<<endl`
 `;`

12

13 `return 0;`

14 `}`

```
1  /*New Keyword*/
2
3  /*Another example program for pointers and
   the use of a "new" keyword is shown below
   .*/
4  #include<iostream>
5  using namespace std;
6
7  int main(){
8      |
9      |     float *p = new float(40.78);
10     |     cout << "The value at address p is " <<
      |         *(p) << endl;
11     |
12     |     return 0;
13 }
14 /*
15 1>We created a float pointer "p" and
   dynamically created a float which has
   value "40.78" and assigned that value to
   pointer "p"
16 2>And printed the value at the address of
   pointer "p"
```

```
1  /*Another example program for pointers array
   and the use of a "new" keyword with an
   array is shown below.,*/
2  #include<iostream>
3  using namespace std;
4
5  int main(){
6
7      int *arr = new int[3];
8      arr[0] = 10;
9      arr[1] = 20;
10     arr[2] = 30;
11     cout << "The value of arr[0] is " <<
        arr[0] << endl;
12     cout << "The value of arr[1] is " <<
        arr[1] << endl;
13     cout << "The value of arr[2] is " <<
        arr[2] << endl;
14
15     return 0;
16 }
17 //point 2B( noted[P2BN]
18 /*we are not giving any name to array, but
   using pointer name to access array
```

1 ▾ `/*Delete Keyword`

2 Another example program for pointers array
and the use of the "delete" keyword with
an array is shown below*/

3 `#include<iostream>`

4 `using namespace std;`

5

6 ▾ `int main(){`

7

8 `int *arr = new int[3];`

9 `arr[0] = 10;`

10 `arr[1] = 20;`

11 `arr[2] = 30;`

12 `delete[] arr;//deletes heap of array`

13 `cout << "The value of arr[0] is " <<
arr[0] << endl;`

14 `cout << "The value of arr[1] is " <<
arr[1] << endl;`

15 `cout << "The value of arr[2] is " <<
arr[2] << endl;`

16

17 `return 0;`

18 `}`

19 ▾ `/*we get the garbage value in the output
becoz pointer is no more pointing to array
elements`

Pointer to objects in C++

As discussed before pointers are used to store addresses of variables which have data types like int, float, double etc. But pointer can also store the address of an object. An example program is shown below to demonstrate the concept of pointer to objects.

```

1  #include<iostream>
2  using namespace std;
3  class Complex{
4      int real, imaginary;
5      public:
6      void getData(){
7          cout<<"The real part is "<< real
8              <<endl;
9          cout<<"The imaginary part is "<<
10             imaginary<<endl;
11         }
12         void setData(int a, int b){
13             real = a;
14             imaginary = b;
15         }
16     };
17     int main(){
18         Complex *ptr = new Complex;
19         (*ptr).setData(1, 54); is exactly same
20             as
21         (*ptr).getData(); is as good as
22         return 0;
23     }
24     //Complex is nothing buy userdefined
25     datatype, so we can write it like this
26     //Now ptr is pointing to an object

```

no name 😊

As shown in a code snippet 1,

1. We created a class “Complex”, which contains two private data members “real” and “imaginary”.
2. The class “complex” contains two member functions “getdata” and “setdata”
3. The Function “setdata” will take two parameters and assign the values of parameters to the private data members “real” and “imaginary”
4. The Function “getdata” will print the values of private data members “real” and “imaginary”

5. In the main program object is created dynamically by using the “new” keyword and its address is assigned to the pointer “ptr”
6. The member function “setdata” is called using the pointer “ptr” and the values “1, 54” are passed.
7. The member function “getdata” is called using the pointer “ptr” and it will print the values of data members.

The main thing to note here is that we called the member function with pointers instead of object but still it will give same result because pointer is pointing to the address of that object.

▼ `/*Arrow Operator in C++`

Another example program for the pointer to Objects and the use of the "Arrow" Operator is shown below.*/

```
4 ▼ int main(){
5     Complex *ptr = new Complex;
6     ptr->setData(1, 54);
7     ptr->getData();
8
9     // Array of Objects
10    Complex *ptr1 = new Complex[4];
11    ptr1->setData(1, 4);
12    ptr1->getData();
13    return 0;
14 }
```

1. We created a class “Complex”, which contains two private data members “real” and “imaginary”.
2. The class “complex” contains two member functions “getdata” and “setdata”
3. The Function “setdata” will take two parameters and assign the values of parameters to the private data members “real” and “imaginary”
4. The Function “getdata” will print the values of private data members “real” and “imaginary”
5. In the main program object is created dynamically by using the “new” keyword and its address is assigned to the pointer “ptr”

6. The member function “setdata” is called using the pointer “ptr” with the arrow operator “->” and the values “1, 54” are passed.
7. The member function “getdata” is called using the pointer “ptr” with the arrow operator “->” and it will print the values of data members.
8. Array of objects is created dynamically by using the “new” keyword and its address is assigned to the pointer “ptr1”
9. The member function “setdata” is called using the pointer “ptr1” with the arrow operator “->” and the values “1, 4” are passed.

10. The member function “getdata” is called using the pointer “ptr1” with the arrow operator “->” and it will print the values of data members.

The main thing to note here is that we called the member function with pointers by using arrow operator “->” instead of the dot operator “.” but still it will give the same results.

The output of the following program is shown below,

Array of Objects Using Pointers in C++

Array of objects can be defined as an array that's each element is an object of the class. In this tutorial, we will use the pointer to store the address of an array of objects. An example program is shown below to demonstrate the concept of an array of objects using pointers.

1. We created a class “ShopItem”, which contains two private data members “id” and “price”.
 2. The class “ShopItem” contains two member functions “setdata” and “getdata”
 3. The Function “setdata” will take two parameters and assign the values of parameters to the private data members “id” and “price”
 4. The Function “getdata” will print the values of private data members “id” and “price”
-

```
1  int main(){
2      int size = 3;
3      ShopItem *ptr = new ShopItem [size];
4      ShopItem *ptrTemp = ptr;
5      int p, i;
6      float q;
7      for (i = 0; i < size; i++)
8      {
9          cout<<"Enter Id and price of item "
10             << i+1<<endl;
11          cin>>p>>q;
12          // (*ptr).setData(p, q);
13          ptr->setData(p, q);
14          ptr++;
15      }
16      for (i = 0; i < size; i++)
17      {
18          cout<<"Item number: "<<i+1<<endl;
19          ptrTemp->getData();
20          ptrTemp++;
21      }
22      return 0;
23 }
```

1. We created an integer variable “size” and assigned the value “3” to it.
2. Array of objects of size “3” is created dynamically by using the “new” keyword and its address is assigned to the pointer “ptr”
3. The address of pointer “ptr” is assigned to another pointer “ptrTemp”
4. Two integer variables “p” and “i” are declared and one float variable “q” is declared

5. We created a “for” loop which will run till the size of array and will take input for “id” and “price” from user at run time. In this “for” loop “setdata” function is called using pointer “ptr”; the function will set the values of “id” and “price” which user will enter. The value of the pointer “ptr” is incremented by 1 in every iteration of loop.

6. We created another “for” loop which will run till the size of array and will print the number of the item. In this “for” loop “getdata” function is called using pointer “ptr”; the function will print the values of “id” and “price”. The value of the pointer “ptrTemp” is incremented by 1 in every iteration of loop.

The main thing to note here is that in the first “for” loop we are incrementing the value of the pointer “ptr” because it is pointing to the address of array of objects and when loop will run every time the function “setdata” will be called by the different object. If we don’t increment the value of the pointer “ptr” the each time function “setdata” will be called by the same object. Likewise in the second loop we are incrementing the pointer “ptrTemp” so that the function “getdata” could be called by each object in the array. The input and output of the following program is shown below,

The input and output of the following program is shown below,

```
Enter Id and price of item 1
1001
1.1
Enter Id and price of item 2
1002
1.2
Enter Id and price of item 3
1003
3.3
```

Figure 1: Array of Objects Using Pointer
Program Input

```
Item number: 1
Code of this item is 1001
Price of this item is 1.1
Item number: 2
Code of this item is 1002
Price of this item is 1.2
Item number: 3
Code of this item is 1003
Price of this item is 3.3
```